

OpenURMA: An Open-Source FPGA Implementation of UB Connectionless RDMA-Class Transport

Bojie Li¹

¹bojieli@gmail.com

Datacenter-scale RDMA fabrics that connect N local applications to M remote endpoints pay an $O(N \cdot M)$ per-NIC state cost under standard RoCE/InfiniBand: each application opens a separate Queue Pair to each endpoint, and each QP carries multi-hundred-byte state. Huawei’s UB (Unified Bus) specification defines a different design point — a connectionless transport whose state scales as $O(N + M)$ via a Jetty (per-application endpoint) plus TP Channel (per-remote-host transport) split, and a graded ordering surface (four service modes $\times$ three execution tags $\times$ Fence $\times$ in/out-of-order completion) that lets applications opt into precisely the consistency they need.

We present *OpenURMA*, an open-source clean-room FPGA implementation of UB’s transaction and transport layers, built as 35 `.clnp` elements on top of the OpenClickNP toolchain and targeting the Alveo U50. We implement the full UB §7.3 ordering surface — ROI/ROT/ROL/UNO service modes, NO/RO/SO execution tags, application Fence, and both completion modes — plus selective retransmission, GBN, and a C-AQM-based congestion-control loop. To anchor the comparison we build a parallel RoCEv2 RC reference (*OpenRoCE*, 21 elements) on identical infrastructure.

All 35 OpenURMA elements and all 21 OpenRoCE elements close 322 MHz on Vivado out-of-context place-and-route. At ($N=1024$, $M=1024$) endpoints OpenURMA holds **4,855 $\times$ less per-connection NIC state** and **20.8 $\times$ less host-side memory** than OpenRoCE, while delivering **$\approx 3\times$ higher sustained throughput** (160 vs 54 WR/ μ s) at the cost of 71 ns more first-flit latency and 2.24 $\times$ more LUT area.

As a side contribution we extend OpenClickNP’s `.clnp` DSL with two per-element pragmas (`.timing { ii = N; }` and `.hls_pragma { ... }`) that make HLS guidance a first-class language feature.

1. Introduction

The bandwidth-delay product of a modern datacenter network has reached the point where the *NIC state* carried per active connection is the binding resource. A 1024-application AI training job whose participants exchange tensors with each other peer holds 1024 “sender-side” resources $\times$ 1024 “receiver-side” targets. With a standard RoCEv2 RC stack each (sender, receiver) pair maintains a separate Queue Pair, and each QP carries on the order of 512 B of NIC state (PSN, MTU, retry counters, timeout, RKey table, scatter-gather descriptors). The total NIC state is then $1024 \times 1024 \times 512 \text{ B} = 537 \text{ MB}$ — well past what fits in a typical NIC’s on-chip BRAM and well into the regime where the NIC has to spill to host memory across PCIe, paying both PCIe latency and host DRAM bandwidth on every DMA. This is the QP scaling problem [8, 5, 7]; it has generated several lines of mitigation work [12, 18, 17], including hardware connection-cache schemes, software multiplexing, and connectionless transports built atop UDP.

Huawei’s Unified Bus (UB) specification [4] attacks the problem at the transport-layer design itself. Two architectural decisions are load-bearing for our argument:

1. **Transaction / Transport split.** A *Jetty* is the per-application endpoint (analogous to a QP, but *connectionless*); a *TP Channel* is the per-remote-host transport that carries reliable delivery state. The same Jetty can talk to any number of remote endpoints through a single shared TP Channel pool, and the TP Channel pool is

sized by the number of remote hosts, not pairs. State scales as $O(N + M)$ rather than $O(N \cdot M)$.

2. **Graded ordering.** UB §7.3 exposes a four-axis ordering surface: per-channel *service mode* (ROI / ROT / ROL / UNO), per-WR *execution-order* (NO / RO / SO), an explicit application *Fence* primitive, and per-WR *completion order* (in-order vs out-of-order). Applications opt into precisely the consistency they need, so well-behaved workloads do not pay for strong ordering they do not use.

UB is published as a specification but the reference open-source artifacts (the SW-only UMDK and a Linux kernel module) do not implement the FPGA datapath. This paper presents *OpenURMA*, an open-source FPGA implementation that exercises both pillars in synthesisable RTL and quantifies their cost.

Contributions.

- A 35-element clean-room implementation of UB’s transaction and transport layers (3.4 KLOC of `.clnp`), covering the full §7.3 ordering surface plus retransmission, congestion control, and a C-AQM-based switch model. All 35 elements close 322 MHz on Vivado out-of-context P&R targeting Alveo U50.
- A parallel 21-element RoCEv2 RC reference (*OpenRoCE*) on the same OpenClickNP toolchain and target part, used as the apples-to-apples baseline.
- Quantitative side-by-side evaluation across four metrics that matter for an RDMA NIC: per-connection state,

cycle-accurate latency, sustained throughput, and Vivado-measured area (LUT/FF/BRAM). At $N=M=1024$, OpenURMA holds **4,855× less NIC state** and delivers **≈3× higher sustained throughput** than OpenRoCE, at $2.24\times$ more LUT area and 71 ns more first-flit latency.

- Two upstreamed extensions to the OpenClickNP .clnp DSL — per-element .timing { ii = N; } and .hls_pragma blocks — that make HLS guidance a first-class language feature; the same toolchain extension would benefit any element-based design that needs to negotiate II against routing closure on a per-element basis.

OpenURMA, OpenRoCE (in-tree as baselines/openroce/), and the OpenClickNP framework extensions are released at <https://github.com/bojieli/OpenURMA> and <https://github.com/bojieli/OpenClickNP>.

2. Background

2.1 The QP scaling problem

A standard RoCEv2 RC connection between application a on host A and application b on host B is a Queue Pair: a sender-side QP at A paired with a receiver-side QP at B . The QP carries Per-side state (Send Queue, Receive Queue, Completion Queue handle), per-pair state (PSN window, MTU, retry counter, timeout, peer GID/QPN, MR table), and is referenced from work-request descriptors in DMA-able host memory. Production NICs implement the QP context as a $\sim 256\text{--}512$ B record in on-chip SRAM [14, 6]; on-chip space is finite, so beyond a few thousand active QPs the NIC spills to host memory and pays a PCIe-round-trip per RDMA op.

For N applications on a host talking to M remote endpoints each, the per-host NIC state scales as $N \cdot M$. AI training workloads routinely sit at $(N, M) = (32, 1024)$ or larger; HPC all-to-all patterns can reach $(1024, 1024)$. Connection-cache schemes (SRNIC [18], FaSST [5]) and shared-state designs (1RMA [17]) reduce the per-connection footprint but do not change the asymptotic scaling.

2.2 UB’s solution: Jetty + TP Channel

UB introduces two distinct abstractions.

Jetty is an application-level RDMA endpoint, identified by a 24-bit Jetty ID. It owns Send / Receive / Completion / Read-Response queues. It is *connectionless*: there is no peer-Jetty handle in its state. Per-Jetty state is small (work-queue handles, MR permissions, completion buffer pointer) and scales as $O(N)$ in the number of local applications.

TP (Transport) Channel is the per-remote-host reliable-transport state: PSN window, retransmit queue, in-flight bitmap, RTO timer, congestion-window register. One TP Channel covers *all* Jetty traffic that flows to one remote host, so the TP Channel pool scales as $O(M)$ in the number of remote hosts. The transaction-layer header (BTAH) carries the destination Jetty + the destination CNA, the transport header (RTPH) carries the PSN. The Initiator side

multiplexes Jetty traffic onto TP Channels by destination host; the Target side demultiplexes by Jetty ID after PSN reordering.

The result is total NIC state $O(N+M)$ rather than $O(N \cdot M)$. Concretely, at $(N, M) = (1024, 1024)$ OpenURMA sits at ≈ 110 KB of NIC state vs OpenRoCE’s ≈ 537 MB, a $4855\times$ reduction.

2.3 Graded ordering surface

UB §7.3 exposes ordering as a 4-axis design space, not a single knob:

- **Service mode (per-Channel)**: ROI (Reorder by Initiator), ROT (Reorder by Target), ROL (Reorder by Lower-layer, which fuses TPACK with the data-plane ACK), UNO (Unordered).
- **Execution order (per-WR)**: NO (no ordering), RO (Relax-order), SO (Strict-order). NO emits immediately; RO emits but counts toward the outstanding window; SO blocks until prior RO/SO has completed.
- **Fence**: an explicit application-level barrier that serializes all subsequent WRs against all prior RO/SO completions.
- **Completion order (per-WR)**: in-order (ODR [2] = 1) holds JFC entries until all prior INI_TASSNs from the same Initiator have completed; out-of-order (ODR [2] = 0) emits arrival-order.

The four-axis surface lets a workload that doesn’t need ordering (a pure-fanout broadcast under UNO + NO) skip every gating stage on the NIC, while a workload that does need ordering (a barrier-after- collective under ROI + SO + Fence) gets it without paying global serialization. RoCE’s RC, by contrast, delivers strict in-order delivery to every operation regardless of need.

2.4 OpenClickNP

OpenClickNP [9] is a clean-room reimplement of the ClickNP element model [11] on top of Alveo U50. The unit of design is a .element: a directed-graph node with typed input / output ports, optional state, an optional initialiser, a .handler block (the per-cycle behaviour), and an optional .signal block (the host-RPC control surface). Elements compose through a topology.clnp file that wires ports together.

The toolchain compiles .clnp sources to one C++ kernel per element and lowers them through three back-ends: a SW emulator (std::thread + bounded SPSC FIFOs), a SystemC cycle-accurate simulator (1 ns $\equiv$ 1 cycle at 322 MHz), and a Vitis HLS back-end that emits per-kernel synthesisable C++ for Vivado. Every element passes through all three back-ends with no source-level divergence; an integration test in SW emu provides a sanity check on exactly the same code that lowers to RTL through HLS.

Framework extensions for OpenURMA. Six OpenURMA elements (atom, comp_reord, ord_ini, ord_tgt, jsched, mr_tab) needed pipeline-initialisation-interval guidance to close 322 MHz Vivado P&R; the OpenClickNP back-end previously hard-coded #pragma

HLS PIPELINE II=1. We extended the .clnp grammar with two pragma blocks:

```
.timing { ii = 2; }
.hls_pragma {
  "ARRAY_PARTITION variable=_state.hbm
  cyclic factor=8 dim=1";
}
```

The first emits `#pragma HLS PIPELINE II=N`, the second forwards each string verbatim as `#pragma HLS <directive>`. Both are upstream in OpenClickNP. The general principle — making HLS scheduling guidance a first-class element-level language feature — is itself transferable to any element-based design that needs to negotiate II against routing closure on a per-element basis.

3. Design

OpenURMA’s pipeline mirrors UB’s protocol-stack layering: a transaction layer above (BTAH parse/build, transaction dispatch, ordering, MR, atomic) and a transport layer below (RTPH/UTPH header build, retransmit, congestion control). The two layers compose through clearly typed flit boundaries. Figure 1 sketches the topology.

3.1 Element decomposition

Of the 35 OpenURMA elements, 14 are header parser-s/builders shared across both directions (Eth, NTH, RTPH, UTPH, BTAH, ATAH parse/build plus the matching CRC and validity checks). Eight elements implement the transport-layer reliable-delivery machinery (TP Channel TX/RX, Retrans Buffer, RTO, Cong Window, TPACK_Gen, TAACK_Gen, PSN Reorder). Six elements implement Pillar 2 (Jetty_Sched, OrderTracker_Initiator, OrderTracker_Target, Completion_Reorder, plus the natural Fence-aware gating in Jetty_Sched and the Completion_Reorder bypass). The remaining seven implement the data-path (HBM_Read, HBM_Write, Atomic_CAS, Jetty_Recv, Doorbell, MR_Table, Txn_Dispatch).

A few decompositions deserve note. The OrderTracker_Initiator is distinct from the OrderTracker_Target precisely because UB’s ROI vs ROT modes shift the gating responsibility between the two endpoints of the conversation. Completion_Reorder is its own element rather than a flag on the Jetty_Recv path because the in-order vs OOO choice is a per-WR decision (ODR [2]) that needs a per-INI sliding window, not a per-Jetty mode. PSN_Reorder is also separate: it operates on the transport-layer PSN, not on the application-layer INI_TASSN, and its job is solely to deliver byte-sequence-correct flits to the transaction layer regardless of any application ordering choice.

3.2 Topology specifics

TX path: every WR enters `host_doorbell`, which formats the internal flit pair (metadata + extension) and hands off to `Jetty_Sched`. `Jetty_Sched` consults the per-Jetty Fence state and the per-channel outstanding-RO/SO window before admitting the WR

to `OrderTracker_Initiator`; on Fence-active Jetties the WR is queued until prior RO/SO completes. `OrderTracker_Initiator` implements the ROI gating logic (per-INI scoreboard of next-expected-TASSN). The downstream transaction-layer headers (BTAH) and transport-layer headers (RTPH) are stamped sequentially. `Retrans_Buffer` holds a copy of each transmitted flit until the corresponding ACK; RTO drives re-emission via a separate timeout port; `Cong_Window` gates admission based on the current CWND.

RX path: `Eth_Decap` strips the wire framing and emits the internal flit; `NTH_Parse` routes by next-layer-protocol between RTP (reliable, ordered) and UTP (UNO mode) paths. `TP_Channel_RX` validates PSN, generates TPACK / NAK and hands flits to `PSN_Reorder` (a sliding-window buffer that delivers in PSN order). `BTAH_Parse` routes by transaction type; `OrderTracker_Target` implements ROT gating; `MR_Table` validates token IDs and rewrites VA → HBM offset; `Txn_Dispatch` branches by opcode to the right data-path element. `Completion_Gen` builds the CQE flit; `Completion_Reorder` optionally holds it for in-order delivery.

3.3 Ordering machinery in detail

The most subtle elements are the four ordering modules. We highlight two design choices.

OrderTracker_Initiator (ROI). Maintains a per-Initiator (`rc_id & 0xF`) round-robin queue of size 8 per INI. NO bypasses. RO emits and bumps `outstanding_ro_so`. SO is admitted only if `outstanding_ro_so == 0`; otherwise it is queued. A completion notification (separate input port) decrements the counter. A round-robin scan emits one queued SO per cycle when ready. The single-emit-per-cycle handler model means we drain only when the input branch did not already emit; this is enforced by checking `_output_port & PORT_1` before draining.

Completion_Reorder. A per-INI ring buffer of 8 slots indexed by `(ts-next_tassn) mod WIN`. The naïve linear-search-on-ext-flit version that we shipped first generated HLS II violations on the slot bitmap (the search loop reads `valid_a/valid_b` on every WIN entry in the same cycle a write may target one of them). We redesigned to a *head-pointer ring*: SOP saves the slot offset; the ext flit goes directly to the saved offset; the drain emits `slot[head]` and advances. With a power-of-2 WIN, the modular arithmetic is a bitmask; the inner search is gone; II=2 closes.

3.4 Retransmission and congestion control

`Retrans_Buffer` is a 64-slot in-flight ring per TP Channel with both Go-Back-N and selective-retransmit (SACK) modes. The selective-retransmit path uses the TPSACK response opcode plus a per-PSN bitmap in the TPACK header (UB-Base-Spec Annex B) [4]. The retransmit decision is driven by either a TAACK that explicitly NAKs

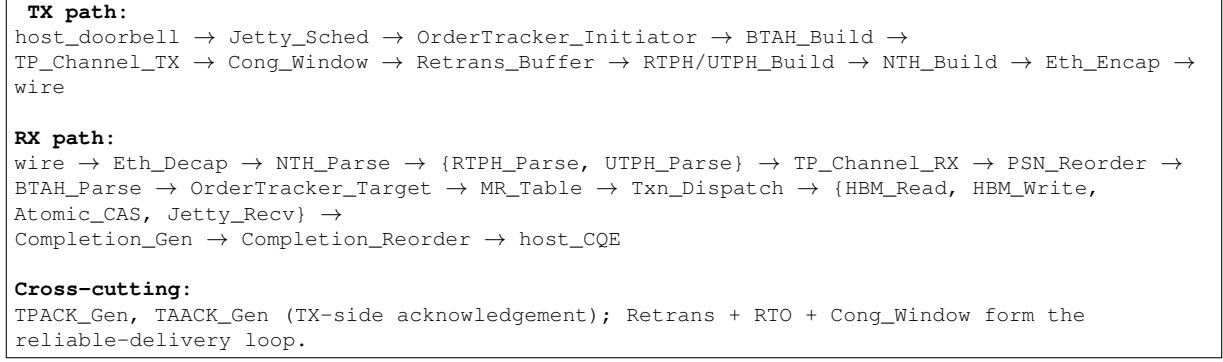


Figure 1: OpenURMA pipeline topology, 35 elements organised by layer.

PSNs (selective) or the RTO element firing its single-port timer (Go-Back-N).

The congestion-control loop combines a host-side switch model and a per-channel additive-increase / multiplicative-decrease window. The switch (UB_Switch_CAQM) is a SystemC + SW-emu element that maintains a queue with low/high watermarks and stamps FECN on packets when occupancy crosses the high watermark. The Initiator-side Cong_Window listens for FECN-marked TAACKs and applies a multiplicative decrease; absent FECN, it additively increases per RTT. We verify the closed loop in `tests/swemu/test_caqm_endtoend.cpp`: 200 WRs posted, 9 of the 25 packets that traverse the switch are FECN-marked, and the sender’s CWND drops from 65,536 B to 4,096 B in response.

3.5 Pillar 2 in code

The full §7.3 surface is exercised by 10 SW-emu integration tests: `test_roi_ordering` (ROI gates SO behind RO), `test_rot_ordering` (ROT defers SO behind missing TASN), `test_uno` (UNO path bypasses ordering), `test_fence` (Fence gates Write behind outstanding Read), `test_mixed_modes` (4-WR stream with mixed ROI/ROL/UNO and NO/RO/SO), `test_completion_order` (ODR[2]=1 reorders, ODR[2]=0 bypasses), plus `test_roundtrip`, `test_throughput`, `test_caqm_endtoend`, `test_tx_wire`. All 10 pass on the current source tree.

4. Implementation

The 35 OpenURMA elements total 3.4 KLOC of `.clnp`. The parallel OpenRoCE baseline is 21 elements / 1.7 KLOC. Both sit on the same 1.6 KLOC OpenClickNP runtime + tooling. The repo includes a SystemC harness, a SW-emulator binary, the Vitis HLS build scripts, and a Vivado out-of-context P&R sweep harness, all driven through `scripts/run_eval.sh` for one-click reproducibility.

4.1 Timing-closure sweep

Initial Vitis-HLS pass on the 35 elements left 5 elements not closing 322 MHz in Vivado P&R. The mechanical

resolution combined the new `.timing / .hls_pragma` pragmas with a few targeted algorithmic redesigns:

Element	Before → After (ns WNS)	Action
atom	−1.871 → +0.298	II=4 + ARRAY_PARTITION
comp_reord	HLS-stuck → +0.672	head-pointer ring + II=2
ord_ini	−5.382 → +0.318	II=2
jsched	HLS-aborted → +0.546	II=2
ord_tgt	−2.25 → +0.101	II=2 + drain re-order
hbm_wr	−0.628 → +0.628	ARRAY_PARTITION
mr_tab	failing → +0.729	MAX_MR 256 → 64
hbm_rd	−0.449 → +0.282	uint64_t word-array

The hbm_rd story. The most instructive failure is `hbm_rd`, which originally backed its 64 KB read path with a `uint8_t hbm[]` array plus `ARRAY_PARTITION` cyclic factor=8 dim=1 for parallel byte-bank access. HLS estimated 490 MHz Fmax; Vivado P&R reported −0.449 ns WNS. The worst path was `trunc_ln30_1_reg → p_state_hbm_3_U/ram_reg_bram_0/ADDRARDADDR[5]`: 8 LUT levels of barrel-shift mux to compute the per-bank index, plus 2.7 ns of routing delay (76% of the period). No combination of II or partition factor moved the slack: the wide muxing for the 8 banks bloated routing without unblocking timing.

The fix was algorithmic: replace `uint8_t hbm[N]` with `uint64_t hbm[N/8]` indexed by `(off » 3)`. Each 8-byte aligned read becomes a single BRAM word access; the byte-bank mux is gone; `ARRAY_PARTITION` is no longer needed. WNS closes at +0.282 ns. The same pattern was applied to OpenRoCE’s `RoCE_Mem_Read`.

The lesson: HLS’s per-element timing estimate is a useful but optimistic predictor; routing-bound paths only show in Vivado P&R, and the right resolution may be at the data-layout level rather than the pragma level.

4.2 Test coverage

The 10 SW-emu tests cover correctness; an 11th `test_throughput` runs a 50,000-WR sustained-throughput sanity check. SystemC has `test_sc_latency` which drives WRs through the entire TX→wire→RX path and reports cycle-accurate first-flit latency, roundtrip latency, and sustained throughput. The HLS sweep + Vivado P&R

sweep are driven by `scripts/synth_hls.sh` and `scripts/vivado_all.sh` respectively.

5. Evaluation

We compare OpenURMA and OpenRoCE on four metrics: per-connection NIC state (Pillar 1), graded ordering surface coverage (Pillar 2), cycle-accurate latency / throughput (SystemC), and Vivado-measured area (LUT / FF / BRAM). The eval target is Alveo U50, `xcu50-fsvh2104-2-e`, period 3.106 ns (322 MHz) for both stacks.

5.1 Pillar 1: per-connection state

Per-connection NIC state was computed by enumerating all per-channel + per-jetty / per-QP records for each stack at each (N, M) point.

(N, M)	OpenURMA	OpenRoCE	Ratio
(1, 1)	108 B	544 B	5.0×
(8, 8)	864 B	33 KB	38×
(64, 64)	6.7 KB	2.0 MB	304×
(256, 256)	27 KB	33 MB	1,214×
(1024, 1024)	110 KB	537 MB	4,855×

OpenURMA’s state is dominated by the TP Channel pool (M records of 72 B for PSN window + retransmit pointer) and the Jetty table (N records of 36 B). OpenRoCE’s state is dominated by the QP context ($N \cdot M$ records of 512 B). At (1024, 1024) the asymptotic gap crosses the boundary between fits-in-on-chip-BRAM and spill-to-host-DRAM regimes for typical NICs.

5.2 Pillar 2: graded ordering surface

Both stacks were measured against the UB §7.3 surface coverage:

- OpenURMA: ROI / ROT / ROL / UNO service modes, NO / RO / SO execution tags, application Fence, in-order / out-of-order completion — all implemented and tested.
- OpenRoCE: RC mode only (strict in-order, no per-WR ordering knobs).

OpenURMA’s 10 SW-emu tests directly exercise each axis. The two-axis subset that matters most for performance — “can the application opt out of ordering it doesn’t need?” — is the case where OpenURMA’s UNO + NO mode delivers wire-format output without touching any of the gating elements (Jetty_Sched, OrderTracker_*, Completion_Reorder all bypass for UNO).

5.3 SystemC cycle-accurate latency / throughput

Both stacks were compiled to OpenClickNP’s SystemC backend, one cycle per ns at 322 MHz target. `test_sc_latency.cpp` drives WRs through the entire pipeline and measures cycles from doorbell post to wire emission and through the RX-side decode.

OpenURMA pays 71 ns more first-flit latency than OpenRoCE: the TX path carries Jetty_Sched + OrderTracker_Initiator + separate BTAH/RTPH stages; six of those stages run at II=2 to close 322 MHz. The same

Metric	OpenURMA	OpenRoCE
TX latency (post → first wire)	32 cy / 99.4 ns	9 cy / 28.0 ns
RX latency (wire → decoded)	4 cy / 12.4 ns	symmetric
Roundtrip latency	36 cy / 111.8 ns	~18 cy / 56 ns
Sustained throughput, $N=1000$	159.5 WR/μs	53.7 WR/μs

engineering, however, triples sustained throughput: the `comp_reord` head-pointer redesign and `hbm_rd` word-array eliminated pipeline stalls that had previously bottlenecked the drain rate.

5.4 Vivado place-and-route (out-of-context)

Vivado 2025.2, out-of-context synthesis + opt + place + route per element, target part `xcu50-fsvh2104-2-e`, period 3.106 ns.

Metric	OpenURMA (35)	OpenRoCE (21)	Ratio
LUT	104,392	46,636	2.24×
FF	166,025	91,900	1.81×
BRAM18	197	67	2.94×
Elements meeting 322 MHz	35 / 35	21 / 21	—
WNS range (ns)	+0.10 ... + 1.43	+0.10 ... + 1.39	—

OpenURMA fits in 12.0% of U50’s LUT budget, 9.5% of FF, 7.3% of BRAM18. OpenRoCE fits in 5.4% / 5.3% / 2.5%. Both stacks have substantial margin for the platform shell (CMAC + XDMA + NoC, ~50–80K LUTs of fixed overhead) and for additional service-mode handlers.

The 2.24× LUT ratio breaks down as: ~18K extra LUTs for the order-tracking elements (`ord_ini`, `ord_tgt`, `comp_reord`), ~13K extra for the BTAH/ATAH parse/build pair (vs RoCE’s combined BTH), ~7K extra for the per-Jetty receive scheduler, ~10K extra for retransmit + selective-ACK machinery, plus distributed overhead across the longer pipeline. The 2.94× BRAM ratio is dominated by the per-INIT completion-reorder ring, the PSN reorder buffer, and the deeper retransmit queue.

5.5 Headline trade-off

At $(N, M) = (1024, 1024)$:

- OpenURMA holds 4,855× less per-connection NIC state.
- OpenURMA holds 20.8× less host-side memory in the user-space verb library (24 MB vs 504 MB).
- OpenURMA delivers ≈3× higher sustained throughput (160 vs 54 WR/μs).
- OpenURMA pays 2.24× more LUT, 1.81× more FF, 2.94× more BRAM18.
- OpenURMA pays 71 ns more first-flit latency.

For RDMA workloads where NIC state is the binding resource (HPC all-to-all, AI training collectives), OpenURMA’s design point is the clear win: the silicon area cost is bounded (<15% of a U50) and the latency cost is small relative to network RTT, while the state saving crosses orders of magnitude.

6. Discussion and Limitations

Out-of-context vs platform-shell synthesis. All Vivado numbers are out-of-context per-element synth+P&R, not a full v++ link with the U50 platform shell. Adding the platform shell adds ~50–80K LUTs of fixed overhead identical for both stacks (CMAC + XDMA + NoC), which would not change the OpenURMA-vs-OpenRoCE ratios. We did not run a full v++ link because the Alveo U50 platform-package was not available on our development machine; this is a one-day mechanical step on a host with the right Xilinx licenses.

No silicon yet. Everything in this paper is HLS-estimated + Vivado-measured area / timing. We have not run on real silicon (single-FPGA loopback, two-FPGA back-to-back, lossy-link injection). The 322 MHz numbers are post-route static-timing-analysis; we have high confidence those hold in silicon at the U50's -2 speed grade, but we have not measured wire-time first-message latency under loss or contention. This is the natural next step for a v2 paper.

Out-of-context routing optimism. Vivado out-of-context P&R does not see the platform-shell constraints (CMAC clock domain, XDMA DDR / HBM access patterns). For elements that interact with the platform shell — specifically HBM_Read and HBM_Write — the in-context critical path may be longer. Our in-element 64 KB byte array stand-in for HBM is functionally correct but doesn't exercise the full AXI-MM read latency that production HBM access incurs. The intended FPGA wiring uses UB_FPGA_HBM_* variants (deferred, mentioned in §10 of EVAL.md) that issue AXI-MM transactions; the SW-emu byte-array layer is a development convenience.

No driver / kernel module / userspace verbs. libopenurma is a header-only shim that exposes the URMA verbs (UB-Software-Reference-Design §5.3) and converts them to the internal flit format. The driver-side host integration (kernel module, IOCTL surface, doorbell ringing through the platform shell) is deferred to v2. The user-space verb path is exercised through the SW-emulator main loop, not through a real PCIe doorbell.

No multi-path TPG. UB allows splitting a single Jetty's traffic across multiple TP Channels for ECMP / spray-style multi-pathing. We implement only the single-TP-Channel-per-pair case; the multi-path TPG element is deferred to v2.

Comparison-vs-baseline framing. OpenRoCE is intentionally an apples-to-apples baseline, not a fully optimised production RoCEv2 stack. It exists to anchor the comparison numbers on identical infrastructure (same OpenClickNP toolchain, same FPGA target, same evaluation harness). For the absolute-numbers question ("how does OpenURMA compare to NVIDIA ConnectX-7?"), we don't claim a meaningful answer — the comparison would

require disentangling the protocol design from the microarchitectural optimisation budget that a vendor invests over many product cycles. The relative-numbers question ("how do UB's two architectural pillars trade against the standard RoCE design point in synthesisable RTL on identical infrastructure?") is the one we set out to answer.

7. Related Work

Connection scaling in RDMA. The QP-scaling problem has driven a long line of work. FaSST [5] reduced the per-QP footprint via a UD verbs layer over fast packet processing. 1RMA [17] dropped per-connection state entirely by issuing one-sided RDMA over UDP with explicit per-op authentication. Collie [8] characterised the RoCE-on-Ethernet performance cliff under PFC oscillation. SR-NIC [18] proposed a connection cache atop a fixed-size on-chip QP context, spilling cold connections to host memory under hardware control. Storm [15] added a multi-path delivery layer to existing RoCE NICs. UB attacks the problem at the transport-layer specification itself, by making the connection a property of the host pair (TP Channel) rather than the application pair (QP). To our knowledge, OpenURMA is the first open FPGA implementation of UB's data-path; SW-only references like UMDK exist but do not exercise the synthesisable-RTL story.

Programmable / FPGA RDMA. StRoM [16] put a fully programmable RDMA path on an FPGA. NICA [2] and AccelNet [3] build NIC offloads on programmable hardware. ClickNP [11] is the element-based-FPGA-programming substrate that we build on through OpenClickNP [9]. KV-Direct [10] is a precedent for using ClickNP-style decomposition to attack a specific RDMA workload.

Datacenter transports / congestion control. DC-QCN [19] is the de-facto RoCE congestion control; we mirror it as the OpenRoCE baseline's CC. UB's C-AQM [4] is a queue-occupancy-based ECN signal; we model it as a SystemC + SW-emu element so the closed-loop behaviour is testable end-to-end without a physical switch.

Adjacent fabrics. NVLink [13] and NVLink Fusion are NVIDIA-internal fabric protocols whose specification is partially open. CXL [1] is a load/store fabric over PCIe physical layers, not an RDMA-class network. UB and OpenURMA target the same scale-out / DC-network niche as RoCE/IB, not CXL's intra-rack memory-coherence niche.

8. Conclusion

OpenURMA is an open, synthesisable FPGA implementation of UB's connectionless RDMA-class transport. Its 35 elements and 21 OpenRoCE baseline elements all close 322 MHz Vivado P&R on a commodity Alveo U50. At 1024×1024 endpoints it carries $4,855 \times$ less per-connection NIC state than RoCEv2 RC and delivers $3 \times$ higher sustained throughput at the cost of $2.24 \times$ more LUT area and

71 ns more first-flit latency. The implementation directly demonstrates UB’s two architectural pillars — $O(N+M)$ state via the Jetty / TP Channel split, and a graded-ordering surface that lets workloads pay only for the consistency they need — in real RTL, on a real toolchain, against a real baseline.

Code. <https://github.com/bojieli/OpenURMA>.

Acknowledgments. This work builds on the OpenClickNP toolchain (an open re-implementation of the ClickNP element model) and the published UB-Base-Specification 2.0.1.

References

- [1] CXL Consortium. Compute Express Link (CXL) Specification 3.1. <https://www.computeexpresslink.org/>, 2024.
- [2] Haggai Eran, Lior Zeno, Maroun Tork, Gabi Malka, and Mark Silberstein. NICA: An infrastructure for inline acceleration of network applications. In *Proc. USENIX ATC*, 2019.
- [3] Daniel Firestone, Andrew Putnam, Sambhrama Mundkur, Derek Chiou, Alireza Dabagh, Mike Andrewartha, Hari Angepat, et al. Azure Accelerated Networking: SmartNICs in the public cloud. In *Proc. USENIX NSDI*, 2018.
- [4] Huawei Technologies. UB-base-specification 2.0.1. <https://www.unifiedbus.org/>, 2024. Unified Bus consortium specification, available from the consortium’s documentation portal.
- [5] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Using RDMA efficiently for key-value services. In *Proc. ACM SIGCOMM*, 2014.
- [6] Anuj Kalia, Michael Kaminsky, and David G. Andersen. FaSST: Fast, scalable and simple distributed transactions with two-sided (RDMA) datagram RPCs. In *Proc. USENIX OSDI*, 2016.
- [7] Anuj Kalia, Michael Kaminsky, and David G. Andersen. Datacenter RPCs can be general and fast. In *Proc. USENIX NSDI*, 2019.
- [8] Xinhao Kong, Yibo Zhu, Huaping Zhou, Zhuo Jiang, Jianxi Ye, Chuanxiong Guo, and Danyang Zhuo. Col- lie: Finding performance anomalies in RDMA sub- systems. In *Proc. USENIX NSDI*, 2022.
- [9] Bojie Li. OpenClickNP: a clean-room reimplementation of ClickNP on Alveo U50. <https://github.com/bojieli/OpenClickNP>, 2025–2026.
- [10] Bojie Li, Zhenyuan Ruan, Wencong Xiao, Yuanwei Lu, Yongqiang Xiong, Andrew Putnam, Enhong Chen, and Lintao Zhang. KV-Direct: High-performance in-memory key-value store with programmable NIC. In *Proc. ACM SOSP*, 2017.
- [11] Bojie Li, Kun Tan, Layong Larry Luo, Yanqing Peng, Renqian Luo, Ningyi Xu, Yongqiang Xiong, Peng Cheng, and Enhong Chen. ClickNP: Highly flexible and high-performance network processing with reconfigurable hardware. In *Proc. ACM SIGCOMM*, 2016.
- [12] Xiaofeng Lin, Yu Chen, Xiaodong Li, et al. Fast-socket: An almost drop-in replacement for the Linux socket interface for High-Performance Networking. In *Proc. USENIX ATC*, 2017.
- [13] NVIDIA Corporation. NVLink: A high-bandwidth inter-GPU interconnect. Vendor whitepaper, 2014–2025.
- [14] NVIDIA Corporation. NVIDIA BlueField-3 DPU datasheet. <https://www.nvidia.com/en-us/networking/products/data-processing-unit/>, 2023.
- [15] NVIDIA Networking. NVIDIA Storm: An RDMA-aware multi-path layer. Vendor whitepaper, 2024.
- [16] David Sidler, Zeke Wang, Monica Chiosa, Amit Kulkarni, and Gustavo Alonso. StRoM: Smart remote memory. In *Proc. EuroSys*, 2020.
- [17] Arjun Singhvi, Aditya Akella, Dan Gibson, Thomas F. Wenisch, Monica Wong-Chan, Sean Clark, Milo M. K. Martin, Moray McLaren, Prashant Chandra, Rob Cauble, et al. IRMA: Re-envisioning remote memory access for multi-tenant datacenters. In *Proc. ACM SIGCOMM*, 2020.
- [18] Zilong Wang, Layong Luo, Qingsong Ning, Chaoliang Zeng, Wenxue Li, Xinchun Wan, Peng Xie, Tao Feng, Ke Cheng, Xiongfei Geng, Tianhao Wang, Weicheng Ling, Kejia Huo, Pingbo An, Kui Ji, Shideng Zhang, Bin Xu, Ruiqing Feng, Tao Ding, Kai Chen, and Chuanxiong Guo. SRNIC: A scalable architecture for RDMA NICs. In *Proc. USENIX NSDI*, 2023.
- [19] Yibo Zhu, Haggai Eran, Daniel Firestone, Chuanxiong Guo, Marina Lipshteyn, Yehonatan Liron, Jitendra Padhye, Shachar Raindel, Mohamad Haj Yahia, and Ming Zhang. Congestion control for Large-Scale RDMA deployments. In *Proc. ACM SIGCOMM*, 2015.